OK to use `404` to indicate a missing entity (as in the previous example)? There are endless proponents in both camps, so take your pick!

`Results` and `TypedResults` include methods for all the common status code results you could need, but if you don't want to use them for some reason, you can always set the status code yourself directly on the `HttpResponse`, as in listing 5.4. In fact, the listing shows how to define the entire response manually, including the status code, the content type, and the response body. You won't need to take this manual approach often, but it can be useful in some situations.

Listing 5.4 Writing the response manually using `HttpResponse`

```
using System.Net.Mime
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
WebApplication app = builder.Build();

app.MapGet("/teapot", (HttpResponse response) =>          ❶
{
    response.StatusCode = 418;                            ❷
    response.ContentType = MediaTypeNames.Text.Plain;     ❸
    return response.WriteAsync("I'm a teapot!");          ❹
});

app.Run();
```

❶ Accesses the HttpResponse by including it as a parameter in your endpoint handler

❷ You can set the status code directly on the response.

❸ Defines the content type that will be sent in the response

❹ You can write data to the response stream manually.

`HttpResponse` represents the response that will be sent to the client and is one of the special types that minimal APIs know to inject into your endpoint handlers (instead of trying to create it by deserializing from the request body). You'll learn about the other types you can use in your endpoint handlers in chapter 7.